

SPLUNK OBSERVABILITY CLOUD

Synthetic Testing

*Why proactive checks matter — and the test types that power them
(Browser Tests & API Tests, correlated with RUM and APM)*

THE PROBLEM

Why Synthetic Testing?

RUM only sees traffic when a real user shows up, and APM only sees a trace once a request already arrived. Neither tells you the site is down before a customer does.

Always-On Checks

Scripted requests run on a schedule from external locations — no waiting on real visitors.

Catch Issues First

Failures and slowdowns surface before a real user is impacted, not after.

Full-Stack Correlation

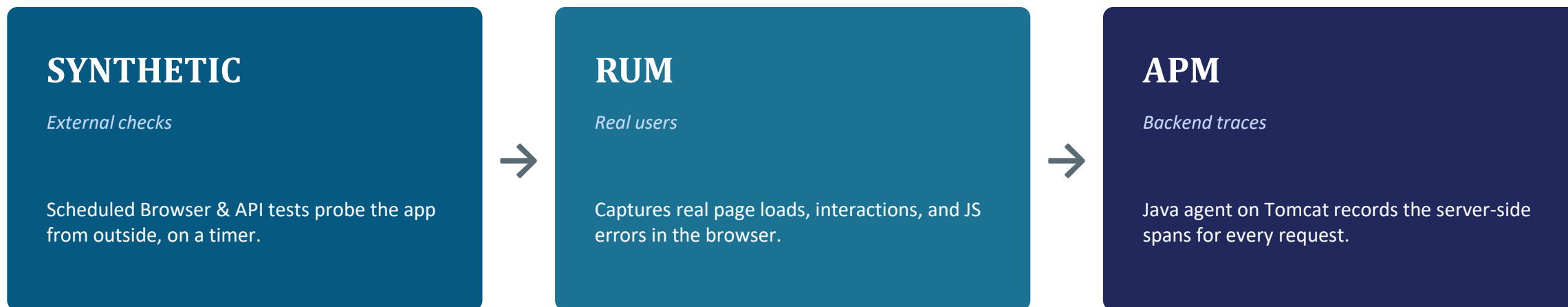
A Server-Timing header links every synthetic run straight to its backend APM trace.

SLA Confidence

Consistent, repeatable checks validate uptime, response time, and content — objectively.

One Question, Three Signals

Because Tomcat already emits real APM traces, wiring up RUM and Synthetics unlocks full three-way correlation — any synthetic run or real browser session can open the exact backend trace that served it.



Linked by a single Server-Timing response header (`SPLUNK_TRACE_RESPONSE_HEADER_ENABLED=true`)

Two Kinds of Synthetic Tests

Browser Tests

Module 6

- Runs a real, scripted browser session
- Navigates, waits, and asserts on page content
- Captures a waterfall of every network request
- Confirms the RUM bundle actually loads
- Verifies the APM link on each request

API Tests (Uptime Checks)

Module 7

- Lightweight HTTP requests — no browser engine
- Checks status code, response time, headers
- Runs far more frequently (as often as every 1 min)
- Can chain multiple requests in one test
- Ideal for cheap, high-frequency health checks

Synthetic Browser Tests

A scripted journey through the actual homepage — steps run in a real browser engine.



Key Configuration

Test Name	Tomcat Lab App — Homepage Journey
Locations	2–3 regions (e.g. us-east-1, eu-west-1)
Frequency	Every 5 minutes

Result: waterfall chart per run, with an APM link on the traced request

Because the Server-Timing header is enabled, clicking a request in the run's waterfall opens the exact tomcat-lab backend trace that handled it — confirming the frontend-to-backend link end to end.

API Test Case Types (Uptime Checks)

Four lightweight, high-frequency checks — each validating a different layer of the stack.

Homepage Availability

Every 1 min

GET / — asserts HTTP 200 and response time under 2000 ms.

RUM Bundle Availability

Every 5 min

GET /splunk-rum-bundle.js — asserts 200 and a JavaScript content-type header.

Server-Timing Header Check

Every 10 min

GET / — asserts 200 and that the response header contains traceparent, proving trace-linking stays configured.

Multi-Step Chained Test

Every 10 min

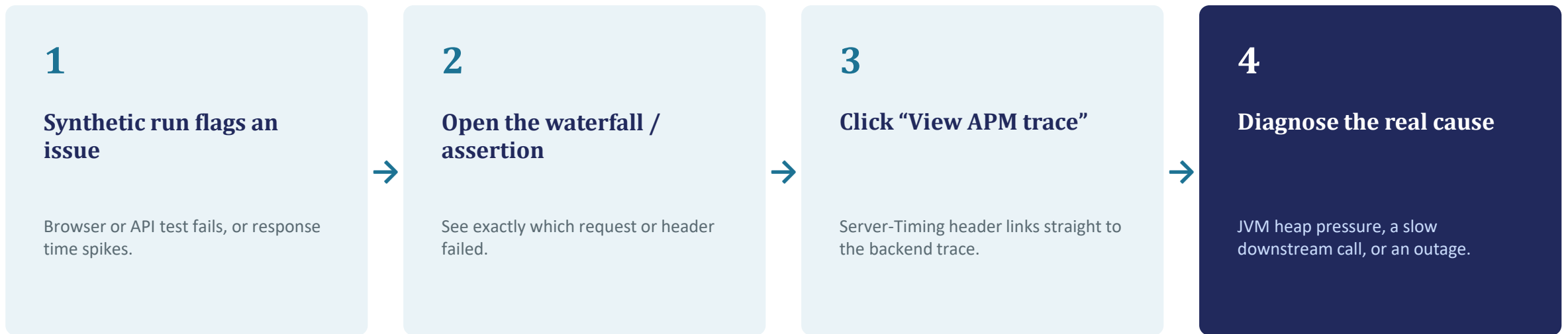
Loads the homepage, extracts an ETag, then loads the RUM bundle — validating a request sequence, not just one URL.

Browser Tests vs. API Tests

Aspect	Browser Test	API Test
Engine	Real browser (renders JS, CSS, DOM)	Plain HTTP request
What it validates	Page content, visual load, RUM bundle presence	Status code, response time, headers, body
Typical frequency	Every 5 minutes	Every 1–10 minutes
Cost / weight	Heavier — full page render	Lightweight — fast and cheap
Best for	User-journey confidence, asset & script checks	High-frequency uptime & header/config checks
APM correlation	Via waterfall request → trace link	Via Server-Timing header assertion

Correlation in Action

A synthetic run's failure or slowdown is never a mystery — it always points to a cause.



Same principle applies to a real RUM session: any XHR/document request can open its matching APM trace.

Key Takeaways

1. Synthetic testing is proactive

It probes the app on a schedule from outside — catching problems before real users hit them.

2. Two complementary test types

Browser Tests validate the real user journey; API Tests give cheap, high-frequency health checks.

3. Four API test patterns covered

Homepage availability, asset availability, header/config validation, and multi-step chains.

4. Every result links to a cause

The Server-Timing header ties synthetic runs (and RUM sessions) directly to APM traces.